2º DAM • CICLO FORMATIVO DE GRADO SUPERIOR

Programación Multimedia y Dispositivos Móviles

Memoria técnica integral y resumen exhaustivo del temario oficial: Entorno y SDKs (Unit 1), Dart y Arquitectura Asíncrona (Unit 2), e Interfaces Declarativas y Navegación con Flutter (Unit 3).

📚 **Módulo:** PMDM (100% Temario) 🔗 **Repositorio:** github.com/arturoalbero/PMDM

📱 **Tecnologías:** Flutter 3.x, Dart 3.x, Material 3, GoRouter

NORMATIVA DE EVALUACIÓN

0. Criterios de Calificación y Memoria Técnica

Según el documento oficial `instrucciones.memoria.md` del repositorio, la memoria no solo es obligatoria para el seguimiento, sino que asienta los conocimientos adquiridos mediante documentación rigurosa:

Elemento de la Memoria	Ponderación	Requisitos Exigidos
Portada	5%	Nombre del alumno, curso (DAM2), fecha de entrega y denominación de la unidad.
Índice	10%	Generado automáticamente mediante jerarquía de encabezados (H2 / H3).
Páginas Numeradas	5%	Paginación uniforme en el documento.
Desarrollo de Actividades	80%	Ordenadas por bloques temáticos, capturas de código relevante, ejecución final, dificultades encontradas y registro de herramientas de asistencia técnica/IA empleadas.

UNIDAD TEMÁTICA 01

1. Entorno de Desarrollo, SDKs y Herramientas

La primera unidad cubre el aprovisionamiento del entorno para el desarrollo multiplataforma profesional (Android, iOS, Web, Windows).

1.1 Visual Studio Code y Productividad

Flutter se integra de forma nativa con VS Code mediante los plugins oficiales de **Dart** y **Flutter**.

Atajos Esenciales

- `Ctrl + Shift + P` : Paleta de comandos (*Flutter: New Project*, *Flutter: Select Device*).
- `Ctrl + .` : Acciones rápidas (Extraer widget, envolver con Padding/Container, convertir a StatefulWidget).
- `r` : *Hot Reload* (recarga en < 500ms preservando el estado de la app).
- `R` : *Hot Restart* (reconstrucción completa reiniciando el estado).

Snippets Clave

- `stless` : Genera la estructura básica de un `StatelessWidget` .
- `stful` : Genera un `StatefulWidget` con su clase de estado asociada.
- `futureBld` : Esqueleto estándar para el widget `FutureBuilder` .

1.2 Dispositivos Físicos, Emuladores y ADB

Para la depuración se emplean dos alternativas de virtualización: el emulador oficial de Android Studio (AVD con aceleración HAXM/Hyper-V) o **Genymotion** (basado en VirtualBox, muy fluido en equipos con recursos moderados).

```
COMANDOS DE DIAGNÓSTICO Y CONTROL DE DISPOSITIVOS

# Verificación general del estado del SDK, herramientas y licencias Android
flutter doctor -v

# Listado de dispositivos reconocidos por el sistema
flutter devices

# Comandos de Android Debug Bridge (ADB)
adb devices              # Lista los terminales y emuladores conectados
adb kill-server          # Reinicia el servidor ADB en caso de cuelgues
adb logcat               # Muestra el flujo de logs en tiempo real del dispositivo
```

1.3 Anatomía Integral de un Proyecto Flutter

- `lib/` : Directorio donde reside el 100% de la lógica Dart y la interfaz. El punto de entrada indiscutible es `lib/main.dart` .
- `pubspec.yaml` : Manifiesto central donde se configuran los metadatos, versiones del SDK, dependencias (`dependencies` / `dev_dependencies`), fuentes y recursos estáticos (imágenes/assets).
- `analysis_options.yaml` : Reglas del linter oficial (`flutter_lints`) para asegurar buenas prácticas y código limpio.
- `android/` , `ios/` , `web/` , `windows/` : Capas puente (*host runners*) que permiten compilar código máquina nativo para cada plataforma.

UNIDAD TEMÁTICA 02

2. Lenguaje Dart, POO y Arquitectura de Datos HTTP

Dart es el lenguaje tipado, orientado a objetos y de alto rendimiento que impulsa Flutter.

2.1 Variables, Tipos y Sound Null Safety

Dart implementa *Sound Null Safety* en tiempo de compilación. Ninguna variable puede contener `null` a menos que se declare explícitamente como anulable.

Operador / Modificador	Significado	Ejemplo de Código
`final` vs `const`	`final` se asigna una sola vez en tiempo de ejecución. `const` es inmutable en tiempo de compilación.	`final now = DateTime.now(); const pi = 3.1416;`
`late`	Inicialización perezosa de una variable no anulable antes de su primer uso.	`late Future<String> _datos;`
Tipo Anulable (`?`)	Permite que la variable admita tanto el tipo como el valor `null`.	`String? nombre;`
Acceso Seguro (`?.`)	Evalúa la propiedad o método solo si el objeto no es nulo; si lo es, retorna nulo sin lanzar excepción.	`int? longitud = nombre?.length;`
Coalescencia Nula (`??`)	Proporciona un valor por defecto si la expresión izquierda resulta nula.	`String texto = nombre ?? 'Anónimo';`
Aserción no nula (`!`)	Fuerza al compilador a tratar la variable como no nula (lanza excepción si es nula).	`int longitud = nombre!.length;`

2.2 Funciones, Parámetros y Operaciones Funcionales

En Dart las funciones son ciudadanos de primera clase y admiten parámetros posicionales y nombrados:

DART: PARÁMETROS NOMBRADOS Y PROGRAMACIÓN FUNCIONAL

```
// Parámetros nombrados obligatorios (required) y opcionales con valor por defecto
void configurarUsuario({required String id, String rol = 'invitado', int? edad}) {
  print('Usuario $id con rol $rol');
}

// Operaciones funcionales en colecciones (List, Set, Map)
final numeros = [1, 2, 3, 4, 5, 6];
final pares = numeros.where((n) => n % 2 == 0).toList(); // [2, 4, 6]
final duplicados = pares.map((n) => n * 2).toList();     // [4, 8, 12]
final total = duplicados.fold(0, (acc, n) => acc + n);   // 24

// Spread operator (...) y collection if/for
final extra = [100, if (total > 20) ...duplicados];
```

2.3 Programación Orientada a Objetos: Factory, Interfaces y Mixins

- **Constructores con Nombre:** Permiten definir múltiples puntos de inicialización (`Usuario.desdeAdmin( ... )`).
- **Constructores Factory:** Utilizan la palabra clave `factory` para retornar instancias cacheadas o deserializar modelos JSON sin crear necesariamente una instancia nueva directa.
- **Clases Abstractas e Interfaces Implícitas:** En Dart toda clase define implícitamente una interfaz. Con `implements` es obligatorio sobreescribir todos sus miembros.
- **Mixins:** Reutilización de código entre jerarquías disjuntas usando `with` (p.ej. `class Perro extends Mamifero with Nadador`).

2.4 Programación Asíncrona: Event Loop, Futures y Streams

Dart se ejecuta en un único hilo con un **Event Loop** respaldado por dos colas: la *Microtask Queue* (alta prioridad) y la *Event Queue* (eventos de I/O, timers, clicks).

DART: ASYNC/AWAIT Y CONSUMO DE STREAMS

```dart
// Operación asíncrona que retorna un Future
Future<String> cargarDatosRemotos() async {
  await Future.delayed(const Duration(seconds: 1));
  return 'Respuesta desde el servidor';
}

// Flujo continuo de eventos con Stream
Stream<int> contadorContinuo() async* {
  for (int i = 1; i <= 3; i++) {
    await Future.delayed(const Duration(seconds: 1));
    yield i; // Emite un nuevo valor en el stream
  }
}
```

2.5 Consumo de APIs HTTP y Arquitectura Limpia

En el caso práctico del repositorio (`healthys_app`), se utiliza el paquete oficial `http` y una separación en capas:

- **Domain Layer:** Entidades y modelos inmutables con métodos `fromJson` y `toJson` .
- **Data Layer:** Servicios remotos (`http.get` , `http.post`) con decodificación de `jsonDecode(response.body)` .
- **Repository Layer:** Abstracción que provee los datos a los widgets aislando el origen de red o caché local.

UNIDAD TEMÁTICA 03 (NÚCLEO PRÁCTICO)

3. Interfaces Declarativas y Navegación en Flutter

La Unidad 3 aborda la creación de interfaces modernas mediante la composición de widgets y los modelos de enrutamiento y transición.

3.1 El Paradigma Declarativo: Widgets y Ciclo de Vida

A diferencia del estilo imperativo tradicional (donde se busca un elemento en el DOM/XML y se muta directamente), en Flutter rige la fórmula:

> **UI = f(State)**
> La interfaz de usuario es una representación pura del estado actual de la aplicación. Cuando el estado cambia mediante `setState()`, el framework reconstruye y renderiza la interfaz calculando las diferencias mínimas sobre el árbol de renderizado.

StatelessWidget

Widget inmutable cuyo aspecto visual depende únicamente de los argumentos recibidos en su constructor. No almacena estado interno mutable.

Método principal: `build(BuildContext context)`.

StatefulWidget

Se compone de dos clases: la del widget (inmutable, que implementa `createState()`) y la del estado (`_MyState`) que perdura y almacena datos mutables.

Ciclo de vida: `initState()` → `build()` → `dispose()`.

★ ACTIVIDAD OFICIAL RESUELTA (Unit 03 - 01): Contador Extendido
Enunciado oficial del repositorio:
"Añadir otro botón para decrementar el contador y otro botón para reiniciar su valor a 0."

Solución arquitectónica implementada en `unit3_activities_app`:
La propiedad `floatingActionButton` del `Scaffold` se compone de un contenedor `Row` con 3 instancias de `FloatingActionButton.extended` (Restar, Reiniciar, Sumar), con sus

respectivos `heroTag` únicos y posicionados en el centro inferior mediante `FloatingActionButtonLocation.centerFloat`.

FRAGMENTO CLAVE: SOLUCIÓN DEL CONTADOR TRIPLE (LIB/SCREENS/COUNTER_ACTIVITY_SCREEN.DART)

```
floatingActionButtonLocation: FloatingActionButtonLocation.centerFloat,
floatingActionButton: Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    FloatingActionButton.extended(
      heroTag: 'btn_dec',
      onPressed: () => setState(() => _counter--),
      label: const Text('Restar (-1)'),
      icon: const Icon(Icons.remove),
    ),
    const SizedBox(width: 10),
    FloatingActionButton.extended(
      heroTag: 'btn_reset',
      onPressed: () => setState(() => _counter = 0),
      label: const Text('Reiniciar (0)'),
      icon: const Icon(Icons.refresh),
    ),
    const SizedBox(width: 10),
    FloatingActionButton.extended(
      heroTag: 'btn_inc',
      onPressed: () => setState(() => _counter++),
      label: const Text('Sumar (+1)'),
      icon: const Icon(Icons.add),
    ),
  ],
)
```

3.2 FutureBuilder y el Patrón Pull-to-Refresh

`FutureBuilder<T>` es el widget estándar para vincular la UI al ciclo de vida de un `Future`.

> ⚠️ **Anti-patrón Crítico señalado en los apuntes:**
> Nunca invoques la función asíncrona directamente dentro del método `build()` (ej: `future: obtenerDatos()`). Dado que `build()` se ejecuta ante cualquier animación o

repintado, la petición se dispararía en bucle. **La regla de oro es instanciar el Future una sola vez en el `initState()` de un StatefulWidget.**

MANEJO RIGUROSO DE CONNECTIONSTATE Y REFRESHINDICATOR

```
@override
void initState() {
  super.initState();
  _itemsFuture = _fetchApiData(); // Se inicializa solo una vez
}

Future<void> _onRefresh() async {
  setState(() {
    _itemsFuture = _fetchApiData(); // Recarga explícita bajo demanda
  });
}

@override
Widget build(BuildContext context) {
  return RefreshIndicator(
    onRefresh: _onRefresh,
    child: FutureBuilder<List<Item>>(
      future: _itemsFuture,
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Center(child: CircularProgressIndicator());
        }
        if (snapshot.hasError) {
          return Center(child: Text('Error: ${snapshot.error}'));
        }
        if (snapshot.hasData) {
          return ListView.builder(...);
        }
        return const SizedBox.shrink();
      },
    ),
  );
}
```

3.3 Navegación Clásica (Navigator 1.0), Formularios y Retorno de Valores

Navigator gestiona una pila LIFO (*Last In, First Out*) de pantallas (*Routes*):

- **Abrir pantalla:** `Navigator.push(context, MaterialPageRoute(builder: (context) ⇒ PantallaDetalle(datos)))`.
- **Cerrar pantalla:** `Navigator.pop(context, resultado)`.
- **Seguridad de pila:** `if (Navigator.canPop(context)) Navigator.pop(context);` evita dejar la aplicación con pantalla en negro si solo queda una ruta en la pila.
- **Retorno Asíncrono:** La llamada a `Navigator.push` es asíncrona. Al utilizar `final res = await Navigator.push( ... )`, la pantalla de origen queda a la espera y captura el resultado transmitido en el `Navigator.pop()`.
- **Validación con Form:** Se asocia un `GlobalKey<FormState>` al widget `Form` y se ejecuta `_formKey.currentState!.validate()` para activar las reglas de los `TextFormField`.

3.4 Widgets de Navegación Material 3 y Diálogos

Estructura de Navegación

- **NavigationBar:** Barra inferior estándar de Material 3 con `NavigationDestination` para terminales móviles.
- **NavigationRail:** Barra lateral en columna recomendada para tablets, monitores y modo horizontal.
- **NavigationDrawer:** Menú desplegable lateral tradicional.
- **TabBar y TabBarView:** Navegación secundaria por pestañas vinculadas mediante un `DefaultTabController`.

Catálogo de Diálogos (showDialog)

- **AlertDialog:** Diálogo con icono, título, contenido y botones de acción (`TextButton`) para confirmaciones.
- **SimpleDialog:** Presenta una lista de alternativas con `SimpleDialogOption`.
- **Dialog:** Ventana emergente con contenido totalmente personalizable (formularios, imágenes).

3.5 Navegación Avanzada: GoRouter y Animaciones Hero

En proyectos medianos/grandes y entornos Web se utiliza **GoRouter**, una biblioteca oficial que implementa la API declarativa de Navigator 2.0:

GOROUTER: DEFINICIÓN DE RUTAS DINÁMICAS Y SHELLROUTE

```
final GoRouter router = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(
      path: '/',
      builder: (context, state) => const HomeScreen(),
    ),
    // Ruta dinámica con parámetro de URL y objeto extra
    GoRoute(
      path: '/detalle/:id',
      builder: (context, state) {
        final id = state.pathParameters['id']!;
        final item = state.extra as ItemModel?;
        return DetalleScreen(id: id, item: item);
      },
    ),
  ],
);

// Navegación declarativa desde cualquier widget:
context.push('/detalle/ENT01', extra: miPlato);
```

Animaciones Hero (Transición de Elementos Compartidos):
Al envolver un widget (ej. una imagen o tarjeta) con `Hero(tag: 'id-unico', child: ...)` en la pantalla origen, y colocar otro `Hero` con el **mismo tag** en la pantalla de destino, Flutter interpola automáticamente su posición y tamaño durante la navegación, logrando una transición visualmente cinematográfica.

ENTREGABLE PRÁCTICO DESARROLLADO

4. Nuevo Proyecto: 'unit3_activities_app'

En respuesta al requerimiento del usuario, se ha creado e inicializado un nuevo proyecto Flutter completo en el workspace:

📁 **Ruta en el workspace:** `programacion movil/Programas/unit3_activities_app`

Archivo / Módulo	Responsabilidad en la Unidad 3
`lib/main.dart`	Punto de entrada configurado con `MaterialApp.router`, soporte de te Material 3 (Claro / Oscuro) y enlace
`lib/routes/app_router.dart`	Definición declarativa de todas las r Unidad 3 con `GoRouter` y parámetr (`/detail/:id`).
`lib/screens/home_dashboard_screen.dart`	Cuadro de mando con banner ilustra métricas de la asignatura y tarjetas interactivas de lanzamiento.
`lib/screens/counter_activity_screen.dart`	**Actividad 3.1:** Contador triple con b para sumar, restar y reiniciar a 0 en `FloatingActionButtonLocation.cer`
`lib/screens/future_builder_activity_screen.dart`	**Actividad 3.2:** Petición asíncrona c `FutureBuilder` , `RefreshIndicator` refresh) y selector de errores.
`lib/screens/navigation_form_activity_screen.dart`	**Actividad 3.3:** Formulario con valida `GlobalKey<FormState>` , envío de ar y captura asíncrona de retorno.
`lib/screens/nav_widgets_dialogs_activity_screen.dart`	**Actividad 3.4:** Navegación responsi `NavigationBar` , `NavigationRail` , `NavigationDrawer` , Tabs y diálogos

Archivo / Módulo	Responsabilidad en la Unidad 3
`lib/screens/gorouter_hero_activity_screen.dart`	**Actividad 3.5:** Catálogo de platos gastronómicos con navegación GoR transición compartida con `Hero`.
`test/widget_test.dart`	Batería de pruebas unitarias y de wi ejecutada con éxito (*All tests passe*

COMANDOS PARA EJECUTAR EL PROYECTO EN LOCAL

```
# Acceder al directorio del proyecto creado
cd "programacion movil/Programas/unit3_activities_app"

# Ejecutar el analizador estático oficial (0 advertencias)
flutter analyze

# Ejecutar la suite de pruebas
flutter test

# Iniciar la aplicación en el dispositivo o plataforma deseada
flutter run -d windows    # En aplicación de escritorio Windows
flutter run -d chrome     # En navegador web con Hot Reload
flutter run -d android    # En emulador o smartphone Android
```
